Document Splitting and Embedding

Friday, March 13, 2026 6:01 PM

Recap

RAG → Cut-off data, publicly available data

↓

LLM's can't answer queries on recent data

↓

Proprietary data

↓

application works/deals

↓

Need to add to LLM.

exceeding content ← prompt Engg ← Small amt

Fine training X

economical RAG.

1) Indexing → Convert data to embedding → Numerical vector → Added / Stored in vector database based on query.
2) Retrieval → Retrieve the similar vector / info from the vect database → Similarity metrics → popular: cosine sim
3) Generation

prompt → [user query + { Sim. info obtained from retrival }]

Act as content to answer to user query.

↓ Input

LLM

↓

Generates response

Indexing

⇒ 1) Document loading → Use langchain data loaders

2) Splitting docs → Chunks

3) Documents Embedding

4) Storing embedding → in database

Data → Document. Loaders → List of langchain document objects

Split the list of langchain documents into Chunks.

way to split

Token count → Not popular

Based on the Character Counts

'hello world'
1 2 3 4 5 6 7 8 9 10

↳ Token count ≈ 2
word count ≈ 2
Character Count = 11

Seperator + Delimiter.

by def '\n\n'

We can change '\n'
" "
" "
" , " ----

List of lang chain document obj → Text Splitter → documents / text_strings.

Based on
Char and
Seperator.

[Chunks.]

embedding
OpenAI

Embedding

OpenAI

Gemini